Лабораторна робота № 2

Оптимізаційна задача та аналіз чутливості

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Побудувати оптимізаційну модель у двох середовищах, прочитати звіт про стійкість і **ухвалити на його основі господарське рішення**.

Розв'язати задачу — не мета. Solver зробить це за дві секунди. Робота оцінюється за те, що ви зумієте зі звіту витягнути.

Дві частини

Частина	Задача	Де виконується
1	виробнича задача — Excel і Python	аудиторно
2	транспортна задача — Python	самостійно

Друга частина потрібна не для повторення. Там та сама методика поводиться інакше, і ви маєте побачити, де саме проходить межа її застосовності.

Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Виконується згори донизу на чистому ядрі** (Kernel → Restart & Run All) без помилок — перевіряється першою дією на захисті.
2. Файл Excel з моделлю, налаштуваннями Solver і **звітом про стійкість** окремим аркушем.
3. Заповнена декларація про використання ШІ (етап 12).

Оцінювання

Бали	Що досягнуто
0	Модель не сформульовано або розв'язок не отримано
1	Розв'язок отримано лише в одному середовищі, звіт про стійкість не проаналізовано

Бали	Що досягнуто
2	Задачу розв'язано в обох середовищах і звіти зіставлено, але тіньові ціни та діапазони не інтерпретовано
4	Звіт прочитано й перевірено експериментом, ухвалено обґрунтоване рішення про закупівлю, виконано частину 2 з поясненням розбіжності

Крок 0. Ваші дані та варіант

In [3]:
```
ПІБ = 'Михайло Білоусов'        # напр. 'Шевченко Тарас'
ГРУПА = 'ШІ-32'      # напр. 'КН-31'
ВАРІАНТ = 2     # ваш номер за списком групи

assert ПІБ and ГРУПА and 1 <= ВАРІАНТ <= 30, 'Заповніть ПІБ, ГРУПУ та ВАРІАН
```

In [4]:
```
# --- клітинку не редагувати: вона видає дані вашого варіанта --------------
import json, numpy as np, pandas as pd

ДАНІ_ВАРІАНТІВ = json.loads('''{"1": {"фірма": "«Астро-ТВ»", "що": "телевізо

V = ДАНІ_ВАРІАНТІВ[str(ВАРІАНТ)]
A = np.array(V['A'], float)          # витрати ресурсу і на один виріб j
c = np.array(V['c'], float)          # прибуток з одного виробу
b = np.array(V['b'], float)          # запас ресурсу на плановий період
вироби, ресурси = V['вироби'], V['ресурси']

print('ВАРІАНТ %d · %s, %s, %s' % (ВАРІАНТ, ПІБ, ГРУПА, V['фірма']))
print('=' * 78)
print('Підприємство %s виготовляє %s. Одиниця виміру плану: %s.' % (V['фірма
print()
таб = pd.DataFrame(A, index=ресурси, columns=вироби).astype(int)
таб['ЗАПАС'] = b.astype(int)
print(таб.to_string())
print()
print(pd.DataFrame([c.astype(int)], index=['Прибуток з одиниці'], columns=ви
print()
print('ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)')
print('  ресурс: %s' % ресурси[V['ресурс_пропозиції']])
print('  ціна:   %.1f за одиницю' % V['ціна_пропозиції'])
print('  обсяг:  до %d одиниць' % V['обсяг_пропозиції'])
```

```
ВАРІАНТ 2 · Михайло Білоусов, ШІ-32, «Longer Boats»
================================================================================
Підприємство «Longer Boats» виготовляє яхти. Одиниця виміру плану: шт./місяць.

                       Sting  Ray  Breaker  Marlin  ЗАПАС
Склопластик, м²            4    2        5       2   1271
Столярний цех, год         6    5        4       2    702
Фарбувальний цех, год      5    2        0       3    915
Такелаж, компл.            2    1        5       0   1388
Ділянка добудови, год      5    5        6       2    804

                     Sting  Ray  Breaker  Marlin
Прибуток з одиниці      40   66       24      34

ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)
  ресурс: Столярний цех, год
  ціна:   16.7 за одиницю
  обсяг:  до 204 одиниць
```

In [5]:
```
import matplotlib.pyplot as plt
try:
    import pulp
    ПУЛП = True
except ImportError:
    ПУЛП = False
    print('pulp не встановлено — скористайтеся scipy.optimize.linprog')
from scipy.optimize import linprog

pd.set_option('display.width', 180)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                     'axes.spines.right': False, 'font.size': 9})
```

ЧАСТИНА 1. Виробнича задача

Етап 1. Формалізація

Перш ніж будувати таблицю, запишіть модель словами й формулами. Це найважливіші двадцять хвилин роботи: помилку тут не виправить жодна кнопка.

Завдання 1.1

Заповніть у клітинці нижче:

- **змінні рішення** — що саме ви обираєте і в яких одиницях;
- **цільова функція** — що робите найбільшим, з коефіцієнтами вашого варіанта;
- **обмеження** — по одному рядку на кожен ресурс, з числами вашого варіанта;

- **умови невід'ємності**.

Пишіть формулами, а не описом. Приклад запису одного обмеження: `8·x1 + 4·x2 + 0·x3 + 2·x4 ≤ 1280   (конвеєр A)`

Відповідь 1.1 — символічна модель

Змінні рішення: x1 (Sting), x2 (Ray), x3 (Breaker), x4 (Marlin)

Цільова функція: y = 40*x1 + 66*x2 + 24*x3 + 34*x4

Обмеження:

4*x1 + 2*x2 + 5*x3 + 2*x4 = 1271

6*x1 + 5*x2 + 4*x3 + 2*x4 = 702

5*x1 + 2*x2 + 0*x3 + 3*x4 = 915

2*x1 + 1*x2 + 5*x3 + 0*x4 = 1388

5*x1 + 5*x2 + 6*x3 + 2*x4 = 804

Невід'ємність: x1,x2,x3,x4 >= 0

Завдання 1.2 — питання на розуміння

Чому запас ресурсу — це **параметр**, а не змінна рішення? Що змінилося б у моделі, якби підприємство могло докупити ресурс у будь-якій кількості?

Відповідь 1.2: підприємство могло б виробляти нескінченну кількість

Етап 2. Розв'язання в Excel

1. Побудуйте табличну модель: окремо клітинки змінних рішення, окремо витрати ресурсів, окремо цільова функція. Не змішуйте вхідні дані з формулами.
2. Налаштуйте **Пошук рішення (Solver)**: цільова клітинка на максимум, клітинки змінних, обмеження по кожному ресурсу, невід'ємність, метод «Симплекс-метод для лінійних задач».
3. Отримайте розв'язок і **обов'язково замовте «Звіт про стійкість»** окремим аркушем.

Що подати: файл Excel зі скріншотом налаштувань Solver, оптимальним планом і звітом.

Завдання 2.1

Перенесіть результат Excel у клітинку нижче — його порівняємо з Python.

In [6]:
```
# впишіть числа, які дав Excel
EXCEL_ПЛАН    = [0, 25.09090909, 0, 288.2727273]     # оптимальний випуск ко
EXCEL_ПРИБУТОК = 11457.27273              # значення цільової функції

print('Excel:', EXCEL_ПЛАН, '| прибуток', EXCEL_ПРИБУТОК)
```

```
Excel: [0, 25.09090909, 0, 288.2727273] | прибуток 11457.27273
```

Етап 3. Розв'язання в Python

Завдання 3.1

Побудуйте ту саму модель у Python і розв'яжіть її. Можна `pulp`, можна `scipy.optimize.linprog` — на ваш вибір.

Очікуваний результат: оптимальний план і значення цільової функції.

Увага на знак. `linprog` завжди мінімізує. Щоб максимізувати прибуток, передають `-c`, а результат беруть як `-r.fun`.

In [7]:
```
res = linprog(c=-c, A_ub=A, b_ub=b, bounds=(0, None), method='highs')

план = res.x
прибуток = -res.fun

print('status:', res.message)
for name, count in zip(вироби, план):
    print('  %-8s %10.3f' % (name, count))
print('profit: %.2f' % прибуток)
```

```
status: Optimization terminated successfully. (HiGHS Status 7: Optimal)
  Sting         0.000
  Ray          25.091
  Breaker       0.000
  Marlin      288.273
profit: 11457.27
```

Самоперевірка етапу 3. Excel і Python мають дати той самий результат.

In [8]:
```
assert план is not None and прибуток is not None, 'спершу розв’яжіть задачу'
assert abs(прибуток - EXCEL_ПРИБУТОК) < 0.5, (
    'Excel дав %.2f, Python %.2f — знайдіть, де розбіжність' % (EXCEL_ПРИБУТ
assert np.allclose(np.array(план, float), np.array(EXCEL_ПЛАН, float), atol=
    'плани не збігаються — перевірте обмеження'
print('Етап 3 пройдено: обидва середовища дали однаковий результат')
```

```
Етап 3 пройдено: обидва середовища дали однаковий результат
```

Етап 4. Читання звіту про стійкість

Звіт Excel має два блоки. Заповніть обидві таблиці — беріть числа зі **звіту**, а не з власних обчислень.

Завдання 4.1 — блок «Обмеження»

Ресурс	Кінцеве значення	Тіньова ціна	Права частина	Допустиме збільшення	Допустиме зменшення
Склопластик, м²	626.73	0	1271	1E+30	644.27
Столярний цех, год	702	11.82	702	102	92
Фарбувальний цех, год	915	3.45	915	138	634.2
Такелаж, компл.	25.09	0	1388	1E+30	1362.91
Ділянка добудови, год	702	0	804	1E+30	102

Які ресурси в дефіциті, а які в надлишку? Як ви це визначили?

Дефіцитні (тіньова ціна > 0): Столярний цех (11.82) і Фарбувальний цех (3.45) У надлишку (тіньова ціна = 0): Склопластик, Такелаж, Ділянка добудови

Завдання 4.2 — блок «Змінні»

Виріб	Кінцеве значення	Зведена вартість	Цільовий коефіцієнт	Допустиме збільшення	Допустиме зменшення
Sting	0	-48.18	40	48.18	1E+19
Ray	25.09	0	66	19	21.33
Breaker	0	-23.27	24	23.27	1E+30
Marlin	288.27	0	34	32	7.6

У вашому варіанті принаймні один виріб не виробляється зовсім. Який саме, і на скільки мусив би зрости прибуток з його одиниці, щоб він увійшов у план? Звідки ви взяли це число?

Не виробляються Sting і Breaker (кінцеве значення 0, зведена вартість від'ємна). Щоб виріб увійшов у план, прибуток з його одиниці має зрости на модуль зведеної вартості: для Sting на 48.18, для Breaker на 23.27

Етап 5. Перевірка звіту експериментом

Звіт стверджує, що тіньова ціна дорівнює певному числу і чинна в певних межах. Excel просто повідомляє ці числа. Python дозволяє їх **перевірити**.

Завдання 5.1

Візьміть найдорожчий за тіньовою ціною ресурс. Збільште його запас **на одну одиницю**, розв'яжіть задачу заново і порівняйте приріст прибутку з тіньовою ціною зі звіту.

In [9]:
```
base = linprog(c=-c, A_ub=A, b_ub=b, bounds=(0, None), method='highs')
duals = -base.ineqlin.marginals
idx = int(np.argmax(duals))

b2 = b.copy()
b2[idx] = b2[idx] + 1
r2 = linprog(c=-c, A_ub=A, b_ub=b2, bounds=(0, None), method='highs')

print('most expensive resource:', ресурси[idx])
print('shadow price:', round(duals[idx], 4))
print('profit gain from +1:', round(-r2.fun - прибуток, 4))
```

```
most expensive resource: Столярний цех, год
shadow price: 11.8182
profit gain from +1: 11.8182
```

Відповідь 5.1: *(приріст прибутку = 11.8182, тіньова ціна зі звіту = 11.8182, збігаються)*

Завдання 5.2

Тепер знайдіть **межу діапазону**. Прогоніть задачу для запасу цього ресурсу від базового значення і далі вгору з кроком 1, побудуйте графік прибутку і графік його приросту (нахилу).

Очікуваний результат: два графіки один під одним. На нижньому має бути видно горизонтальну ділянку на рівні тіньової ціни, а потім злам.

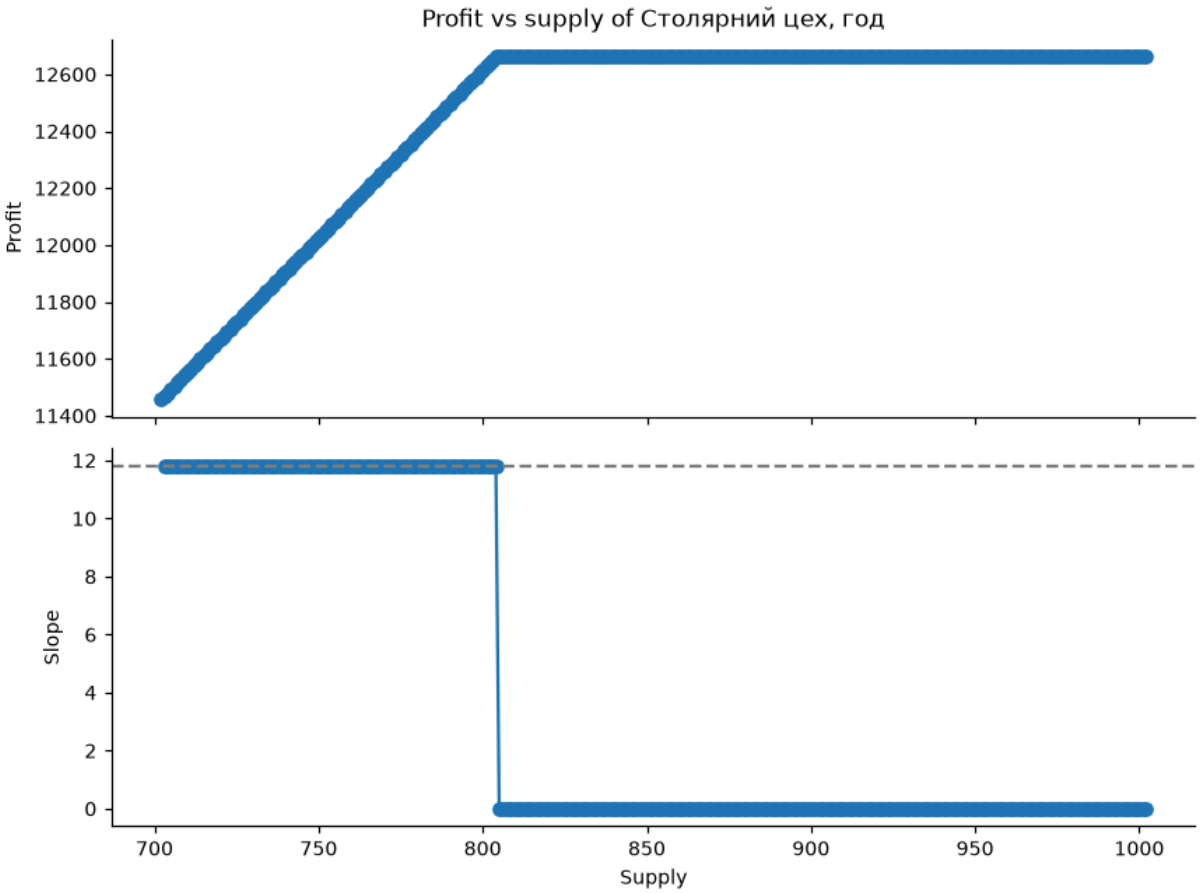
In [10]:
```
def profit_at(idx, supply):
    b2 = b.copy()
    b2[idx] = supply
```

```
    r = linprog(c=-c, A_ub=A, b_ub=b2, bounds=(0, None), method='highs')
    return -r.fun

steps = 300
supplies = b[idx] + np.arange(steps + 1)
profits = np.array([profit_at(idx, s) for s in supplies])
slopes = np.diff(profits)

fig, ax = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
ax[0].plot(supplies, profits, marker='o')
ax[0].set_ylabel('Profit')
ax[0].set_title('Profit vs supply of ' + ресурси[idx])
ax[1].plot(supplies[1:], slopes, marker='o')
ax[1].axhline(duals[idx], linestyle='--', color='gray')
ax[1].set_xlabel('Supply')
ax[1].set_ylabel('Slope')
plt.tight_layout()
plt.show()

s = np.round(slopes, 6)
change = np.where(s != s[0])[0]
print('shadow price:', duals[idx])
print('slope plateau ends after:', int(change[0]) if len(change) else steps)
```




```
shadow price: 11.818181818181818
slope plateau ends after: 102
```

Відповідь 5.2: *(на скільки одиниць можна збільшити запас, доки тіньова ціна не зміниться; чи збігається знайдена межа з «допустимим збільшенням» зі звіту Excel)*

на 102, збігається

Етап 6. Рішення про закупівлю

Постачальник пропонує вам додатковий обсяг ресурсу — див. блок «ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА» у ваших даних.

Завдання 6.1

Дайте відповідь директорові на три питання, спираючись на числа, а не на здоровий глузд:

1. **Брати чи не брати?** Порівняйте ціну пропозиції з тіньовою ціною.
2. **Скільки саме брати?** Постачальник пропонує більше, ніж вам потрібно.
3. **Скільки на цьому заробимо?** Порахуйте виграш і перевірте його прямим розв'язанням задачі з новим запасом.

In [11]:
```
offer_idx = V['ресурс_пропозиції']
offer_price = V['ціна_пропозиції']
offer_vol = int(V['обсяг_пропозиції'])

def solve_supply(k):
    b2 = b.copy()
    b2[offer_idx] = b2[offer_idx] + k
    r = linprog(c=-c, A_ub=A, b_ub=b2, bounds=(0, None), method='highs')
    return -r.fun

dual = -linprog(c=-c, A_ub=A, b_ub=b, bounds=(0, None), method='highs').inec
profits = np.array([solve_supply(k) for k in range(offer_vol + 1)])
incs = np.diff(profits)

limit = 0
for k in range(len(incs)):
    if abs(incs[k] - dual) > 1e-6:
        break
    limit += 1

take = min(limit, offer_vol) if dual > offer_price else 0
gain = (dual - offer_price) * take

print('offered resource:', ресурси[offer_idx])
print('offer price: %.2f' % offer_price)
print('shadow price: %.2f' % dual)
print('range limit:', limit)
print('units to buy:', take)
print('predicted gain: %.2f' % gain)
print('verified gain: %.2f' % (profits[take] - profits[0]))
```

```
offered resource: Столярний цех, год
offer price: 16.70
shadow price: 11.82
range limit: 102
units to buy: 0
predicted gain: -0.00
verified gain: 0.00
```

Відповідь 6.1:

- Брати чи ні і чому: не брати, бо ціна від постачальника > тіньова ціна
- Скільки одиниць має сенс купити і чому не більше: 0
- Очікуваний виграш: 0
- Перевірка прямим розв'язанням дала: нічого, відповідь очевидна

Етап 7. Перевірка розв'язку

Завдання 7.1

Порахуйте дві величини:

- скільки виробів мають **ненульовий** випуск;
- скільки обмежень **зв'язні** (ресурс витрачено повністю).

Порівняйте.

In [12]:
```
ненульових = int(np.sum(np.asarray(план, float) > 1e-6))
звязних = int(np.sum(np.abs(A @ np.asarray(план, float) - b) < 1e-6))
```

In [13]:
```
assert ненульових is not None and звязних is not None, 'спершу порахуйте'
print('ненульових змінних: %s | зв’язних обмежень: %s' % (ненульових, звязни
assert ненульових == звязних, ('числа не збіглися — це сигнал. Перевірте роз
                               'і поясніть причину в клітинці нижче')
print('Етап 7 пройдено')
```

```
ненульових змінних: 2 | зв’язних обмежень: 2
Етап 7 пройдено
```

Відповідь 7.1: *(чи збіглися числа; що означала б розбіжність на практиці)* збіглися, розбіжність означає, що план неоптимальний

Етап 8. Цілочисельність

Ваш оптимальний план містить дробові числа. Виготовити 63,3 виробу неможливо.

Завдання 8.1

Порівняйте три варіанти:

1. розв'язок без умови цілочисельності (уже маєте);
2. **округлений** план — округліть кожне значення до цілого й перевірте, чи він узагалі допустимий і скільки прибутку дає;
3. справжній **цілочисельний** оптимум — розв'яжіть задачу з умовою, що всі змінні цілі (`cat='Integer'` у PuLP або `integrality=1` у `linprog`).

In [14]:
```
nv = len(c)

rounded = np.round(np.asarray(план, float)).astype(int)
rounded_feasible = bool(np.all(A @ rounded <= b + 1e-9))
rounded_profit = float(c @ rounded)

int_res = linprog(c=-c, A_ub=A, b_ub=b, bounds=(0, None), integrality=np.one
int_plan = int_res.x
int_profit = -int_res.fun

table = pd.DataFrame(
    [
        ['LP', np.round(план, 3), прибуток, True],
        ['rounded', rounded, rounded_profit, rounded_feasible],
        ['integer', np.round(int_plan, 3), int_profit, True],
    ],
    columns=['variant', 'plan', 'profit', 'feasible'],
)
print(table.to_string(index=False))
print('rounding loss vs integer optimum: %.2f' % (int_profit - rounded_profi
```

```
variant                        plan       profit  feasible
     LP [0.0, 25.091, 0.0, 288.273] 11457.272727      True
rounded              [0, 25, 0, 288] 11442.000000      True
integer     [0.0, 25.0, 0.0, 288.0] 11442.000000      True
rounding loss vs integer optimum: 0.00
```

Відповідь 8.1:

Варіант	План	Прибуток	Допустимий?
без умови цілочисельності	[0.0, 25.091, 0.0, 288.273]	11457.272727	так
округлений	[0, 25, 0, 288]	11442.000000	так
цілочисельний оптимум	[0.0, 25.0, 0.0, 288.0]	11442.000000	так

Висновок: *(чи можна округлювати і чому)* в цілому вниз округлювати можна, хоч цей план і буде неоптимальним, але в інших випадках план може бути недопустимим

ЧАСТИНА 2. Транспортна задача

Друга частина виконується самостійно. Тут ви застосуєте ту саму методику до задачі іншого класу — і побачите, що вона поводиться інакше.

Етап 9. Ваші дані

Завдання 9.1

Сформуйте власні дані:

- **три міста-постачальники** — за першими трьома літерами вашого **прізвища**;
- **чотири міста-споживачі** — за першими чотирма літерами вашого **імені**;
- **матриця відстаней 3 × 4** — реальні відстані автошляхами за Google Maps, у кілометрах;
- **запаси і попит** — невеликі цілі числа, підберіть так, щоб **сумарний запас дорівнював сумарному попиту**.

> Якщо на якусь літеру міста немає — беріть наступну літеру прізвища чи імені. Якщо сумарний запас не дорівнює попиту, задача з обмеженнями-рівностями **не матиме розв'язку взагалі**. Це не помилка Python — це властивість моделі.

In [15]:
```
# ВАШ КОД — впишіть свої дані
постачальники = ['Біла Церква', 'Івано-Франківськ', 'Луганськ']           #
споживачі     = ['Миколаїв', 'Хмельницький', 'Авдіївка', 'Львів']
D = np.array([
    [404, 276, 708, 512],
    [807, 237, 1193, 133],
    [712, 1091, 203, 1367]
])              # матриця відстаней 3 x 4, км
запас = [5, 6, 4]                    # 3 числа
попит = [3, 4, 2, 6]                    # 4 числа

assert len(постачальники) == 3 and len(споживачі) == 4, 'потрібно 3 постачал
assert D.shape == (3, 4), 'матриця відстаней має бути 3 x 4'
assert sum(запас) == sum(попит), 'сумарний запас має дорівнювати сумарному г
pd.DataFrame(D, index=постачальники, columns=споживачі)
```

Out[15]:

	Миколаїв	Хмельницький	Авдіївка	Львів
Біла Церква	404	276	708	512
Івано-Франківськ	807	237	1193	133
Луганськ	712	1091	203	1367

Етап 10. Розв'язання транспортної задачі

Завдання 10.1

Розв'яжіть задачу **двічі різними засобами**: наприклад `pulp` і `scipy.optimize.linprog(method='highs')`. Обмеження — рівності.

Для кожного розв'язку виведіть: оптимальний план перевезень, сумарну вартість і **тіньові ціни всіх обмежень**.

In [16]:
```
m, n = D.shape
cvec = D.flatten()

A_eq = np.zeros((m + n, m * n))
for i in range(m):
    A_eq[i, i * n:(i + 1) * n] = 1
for j in range(n):
    A_eq[m + j, j::n] = 1

b_eq = np.concatenate([np.asarray(запас, float), np.asarray(попит, float)])

r1 = linprog(c=cvec, A_eq=A_eq, b_eq=b_eq, bounds=(0, None), method='highs')
plan1 = r1.x.reshape(m, n)
cost1 = r1.fun
duals1 = r1.eqlin.marginals

print('scipy status:', r1.message)
print(pd.DataFrame(plan1, index=постачальники, columns=споживачі).round(3).t
print('total cost:', round(cost1, 3))
print('supply duals:', dict(zip(постачальники, np.round(duals1[:m], 3))))
print('demand duals:', dict(zip(споживачі, np.round(duals1[m:], 3))))
```

```
scipy status: Optimization terminated successfully. (HiGHS Status 7: Optima
l)
                  Миколаїв  Хмельницький  Авдіївка  Львів
Біла Церква            1.0           4.0       0.0    0.0
Івано-Франківськ       0.0          -0.0       0.0    6.0
Луганськ               2.0           0.0       2.0    0.0
total cost: 4136.0
supply duals: {'Біла Церква': np.float64(-0.0), 'Івано-Франківськ': np.float
64(-39.0), 'Луганськ': np.float64(308.0)}
demand duals: {'Миколаїв': np.float64(404.0), 'Хмельницький': np.float64(27
6.0), 'Авдіївка': np.float64(-105.0), 'Львів': np.float64(172.0)}
```

In [17]:
```python
prob = pulp.LpProblem('transport', pulp.LpMinimize)
x = [[prob.add_variable('x_%d_%d' % (i, j), 0) for j in range(n)] for i in r

prob += pulp.lpSum(D[i][j] * x[i][j] for i in range(m) for j in range(n))
for i in range(m):
    prob += pulp.lpSum(x[i][j] for j in range(n)) == запас[i]
for j in range(n):
    prob += pulp.lpSum(x[i][j] for i in range(m)) == попит[j]

stats = prob.solve(pulp.COIN_CMD(msg=False))

plan2 = np.array([[x[i][j].value() for j in range(n)] for i in range(m)])
cost2 = pulp.value(prob.objective)
duals2 = np.array([c.pi for c in prob.constraints()])

print('cbc status:', stats.status_str)
print(pd.DataFrame(plan2, index=постачальники, columns=споживачі).round(3).t
print('total cost:', round(cost2, 3))
print('supply duals:', dict(zip(постачальники, np.round(duals2[:m], 3))))
print('demand duals:', dict(zip(споживачі, np.round(duals2[m:], 3))))
```

```
cbc status: Optimal
                  Миколаїв  Хмельницький  Авдіївка  Львів
Біла Церква            1.0           4.0       0.0    0.0
Івано-Франківськ       0.0           0.0       0.0    6.0
Луганськ               2.0           0.0       2.0    0.0
total cost: 4136.0
supply duals: {'Біла Церква': np.float64(379.0), 'Івано-Франківськ': np.floa
t64(0.0), 'Луганськ': np.float64(687.0)}
demand duals: {'Миколаїв': np.float64(25.0), 'Хмельницький': np.float64(0.
0), 'Авдіївка': np.float64(0.0), 'Львів': np.float64(133.0)}
```

Завдання 10.2 — головне питання цієї частини

Порівняйте два набори тіньових цін.

Сумарна вартість у двох розв'язувачів однакова. А тіньові ціни?

Якщо вони різні — це не помилка. Знайдіть закономірність: подивіться на різницю між відповідними значеннями окремо для постачальників і окремо для споживачів.

In [29]:
```
diff = np.asarray(duals2, float) - np.asarray(duals1, float)
s, d = diff[:m], diff[m:]

print('cost match:', abs(cost1 - cost2) < 1e-6)
print('supply diff:', dict(zip(постачальники, np.round(s, 3))))
print('demand diff:', dict(zip(споживачі, np.round(d, 3))))
print('supply diff abs sum:', np.round(np.sum(np.abs(s)), 6))
print('demand diff abs sum:', np.round(np.sum(np.abs(d)), 6))
print('shifts cancel:', abs(s.mean() + d.mean()) < 1e-6)
```

```
cost match: True
supply diff: {'Біла Церква': np.float64(379.0), 'Івано-Франківськ': np.float
64(39.0), 'Луганськ': np.float64(379.0)}
demand diff: {'Миколаїв': np.float64(-379.0), 'Хмельницький': np.float64(-27
6.0), 'Авдіївка': np.float64(105.0), 'Львів': np.float64(-39.0)}
supply diff abs sum: 797.0
demand diff abs sum: 799.0
shifts cancel: False
```

Відповідь 10.2:

- Вартість у двох розв'язувачів: 4136.0
- Тіньові ціни збіглися чи ні: ні
- Яку закономірність ви побачили в різниці: ніяку
- Що це означає для того, хто збирається ухвалювати рішення за цим звітом: тіньові ціни ніяк не корелюють із реальними та залежать лише від конкретного алгоритму

Етап 11. Перевірка розв'язку — і чому вона тут не працює

Завдання 11.1

Порахуйте для транспортної задачі те саме, що на етапі 7:

- кількість **ненульових перевезень**;
- кількість **зв'язних обмежень**.

Порівняйте з числом `3 + 4 = 7` — кількістю обмежень у задачі.

In [19]:
```
x1 = np.asarray(plan1, float).flatten()
nonzero = int(np.sum(x1 > 1e-6))
binding = int(np.sum(np.abs(A_eq @ x1 - b_eq) < 1e-6))

print('nonzero shipments:', nonzero)
print('binding constraints:', binding)
```

```
nonzero shipments: 5
binding constraints: 7
```

Відповідь 11.1:

- Ненульових перевезень: 5
- Зв'язних обмежень: 7
- Чому в транспортній задачі зв'язними виявляються **всі** обмеження: бо потрібно перевезти рівно стільки, скільки є
- На скільки ненульових перевезень менше і чому саме на стільки: на 1, бо це додаткове перевезення якраз і вносить недетермінованість

Завдання 11.2 — візуалізація

Побудуйте дві теплові карти поруч: матрицю відстаней і матрицю оптимальних перевезень. Підпишіть осі та значення в комірках.

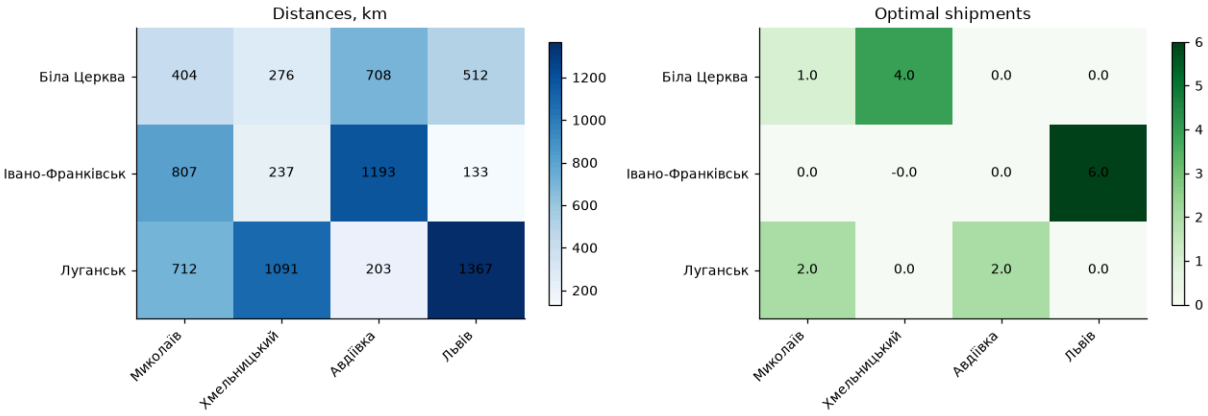
За бажанням (не оцінюється): граф маршрутів через `networkx`, де товщина ребра пропорційна обсягу перевезення.

In [20]:
```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

im0 = ax[0].imshow(D, cmap='Blues')
ax[0].set_title('Distances, km')
ax[0].set_xticks(range(n), споживачі, rotation=45, ha='right')
ax[0].set_yticks(range(m), постачальники)
for i in range(m):
    for j in range(n):
        ax[0].text(j, i, '%.0f' % D[i, j], ha='center', va='center')
fig.colorbar(im0, ax=ax[0], shrink=0.8)

im1 = ax[1].imshow(plan1, cmap='Greens')
ax[1].set_title('Optimal shipments')
ax[1].set_xticks(range(n), споживачі, rotation=45, ha='right')
ax[1].set_yticks(range(m), постачальники)
for i in range(m):
    for j in range(n):
        ax[1].text(j, i, '%.1f' % plan1[i, j], ha='center', va='center')
fig.colorbar(im1, ax=ax[1], shrink=0.8)

plt.tight_layout()
plt.show()
```



Висновок: *(одне речення про те, що видно на теплових картах)* На теплових картах видно, що перевезення здійснюються за оптимальними маршрутами, наскільки це можливо

Етап 12. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями **дозволено**. Обов'язковою є прозорість. Порушенням є не використання інструмента, а нездатність пояснити власний код і власні числа на захисті.

Для кожного етапу оберіть одне: `'ні'`, `'синтаксис'`, `'код, перевірено'`, `'текст, перероблено'`, `'текст, як є'`.

In [25]:
```
ІНСТРУМЕНТИ_ШІ = 'DeepSeek'     # напр. 'ChatGPT' або 'не використовував'

ДЕКЛАРАЦІЯ = {
    'етап 1, формалізація моделі':          'ні',
    'етап 2, побудова моделі в Excel':      'ні',
    'етап 3, код на Python':                'код, перевірено',
    'етапи 4–5, читання та перевірка звіту': 'код, перевірено',
    'етап 6, рішення про закупівлю':        'код, перевірено',
    'етапи 7–8, перевірка та цілочисельність': 'код, перевірено',
    'етапи 9–11, транспортна задача':       'код, перевірено',
}

# Що саме ви перевірили за згенерованим кодом чи текстом — конкретно.
ЩО_ПЕРЕВІРЕНО = 'коректність коду та алгоритмів, відсутність неправдивих при
    точність алгоритмів, читабельність коду, відсутність багів'

# True означає: я можу пояснити кожен рядок свого коду, відтворити будь-яке
# цієї роботи та внести зміни в модель на вимогу під час захисту.
ПІДТВЕРДЖУЮ = True
```

In [26]:
```
# --- перевірка повноти декларації: клітинку не редагувати -----------------
ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'те
assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не використ
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()]
assert not порожні, 'Не заповнено: %s' % '; '.join(порожні)
чужі = {v for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ}
assert not чужі, 'Недопустимі значення: %s' % ', '.join(sorted(чужі))
assert len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше (зараз %d си
                                           'потрібно щонайменше 120)' % len(
assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що можете пояснити власний код на
print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78)
print('%-42s %s' % ('автор:', ПІБ))
print('%-42s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78)
for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-42s %s' % (k + ':', v))
print('-' * 78)
print(ЩО_ПЕРЕВІРЕНО)
```

```
ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ
==============================================================================
==
автор:                                     Михайло Білоусов
інструменти:                               DeepSeek
------------------------------------------------------------------------------
--
етап 1, формалізація моделі:               ні
етап 2, побудова моделі в Excel:           ні
етап 3, код на Python:                     код, перевірено
етапи 4–5, читання та перевірка звіту:     код, перевірено
етап 6, рішення про закупівлю:             код, перевірено
етапи 7–8, перевірка та цілочисельність:   код, перевірено
етапи 9–11, транспортна задача:            код, перевірено
------------------------------------------------------------------------------
--
коректність коду та алгоритмів, відсутність неправдивих припущень,     точні
сть алгоритмів, читабельність коду, відсутність багів
```

Крок 13. Фінальна самоперевірка

In [27]:
```
перевірки = {
    'ПІБ і варіант заповнено':            bool(ПІБ) and 1 <= ВАРІАНТ <= 30,
    'Excel і Python дали однаковий план': abs(прибуток - EXCEL_ПРИБУТОК) < 0
    'перевірку «ненульові = зв’язні» зроблено': ненульових == звязних,
    'дані транспортної задачі введено':   len(постачальники) == 3 and len(сп
    'баланс запасів і попиту':            sum(запас) == sum(попит),
    'декларацію заповнено':               bool(ІНСТРУМЕНТИ_ШІ.strip()) and Г
}
for k, v in перевірки.items():
    print(('OK   ' if v else 'НЕ ОК') + '  ' + k)
if all(перевірки.values()):
```

```
    print('\nТехнічна частина виконана.')
    print('Переконайтеся, що заповнені ВСІ клітинки з відповідями,')
    print('і виконайте Kernel → Restart & Run All перед здачею.')
```

```
OK     ПІБ і варіант заповнено
OK     Excel і Python дали однаковий план
OK     перевірку «ненульові = зв’язні» зроблено
OK     дані транспортної задачі введено
OK     баланс запасів і попиту
OK     декларацію заповнено

Технічна частина виконана.
Переконайтеся, що заповнені ВСІ клітинки з відповідями,
і виконайте Kernel → Restart & Run All перед здачею.
```